Fundamentos de Sistemas Operacionais

Apresentação/Introdução

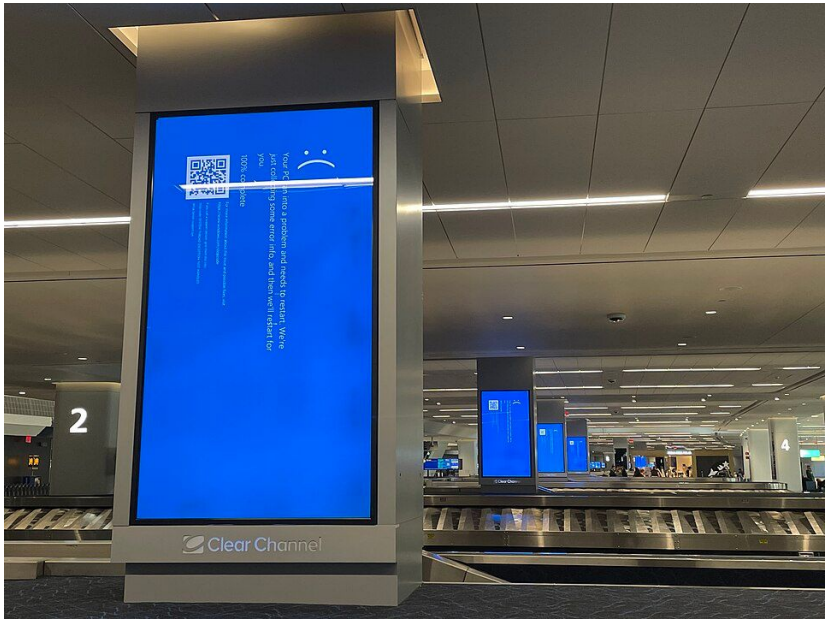
Prof. Msc. Lucas Reis

lucas.oliveira@puc-campinas.edu.br

Campinas, 2s/2026

Objetivos

- **Fundamentos**
  - O que faz um SO, por que é essencial
- **Princípios de Design**
  - Como são desenvolvidos e as vantagens e desvantages de cada tipo
- **Estrutura de um SO**
  - Componentes majoritários e suas funções (processos, memória, armazenamento, segurança, …)
- **Gerenciamento de Recursos**
  - Como um SO gerencia e distribui seus recursos entre usuários e processos



Contato

E-mail

lucas.oliveira@puc-campinas.edu.br

Assunto: [SO - *Turma*]

Canvas

Mensagens diretas

Canvas

- **Material**
  - Slides
    - Se eu esquecer de algo, **não tenham medo de me avisar**
- **Avisos**
- **Notas**

Calendário

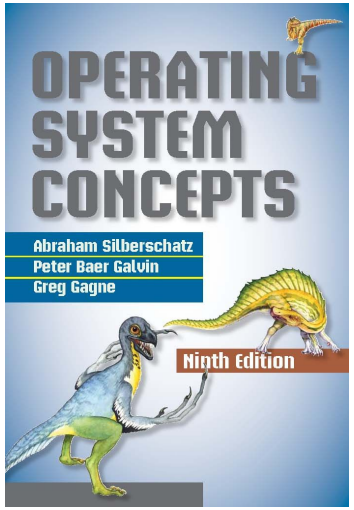
Primeira Parte

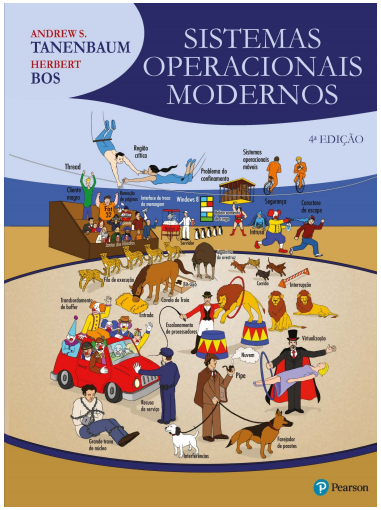
- Introdução
- Processos, threads e sincronização
- Escalonamento e concorrência

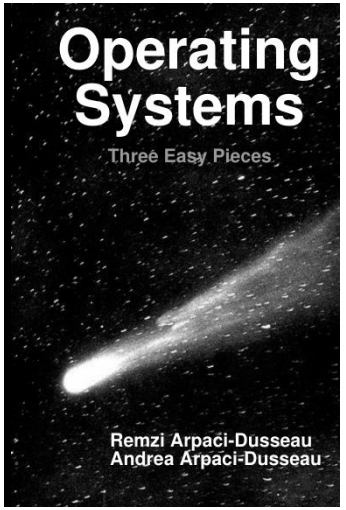
Segunda Parte

- Gerenciamento de Arquivos
- Gerenciamento de Entrada/Saída
- Gerenciamento de Memória Virtual

Bibliografia







Abraham **Silberschatz,** Peter Galvin, Greg Gagne
Fundamentos de Sistemas Operacionais
9a edição

Andrew **Tanenbaum**, herbert Bos
Sistemas Operacionais Modernos
4a edição

Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau
Operating Systems: Three Easy Pieces

Avaliação

Parte 1

- Prova 1 - P1
  - Eng. Soft T03 (seg) - **28/set**
  - Eng. Soft T02 (sex) - **09/out**
  - SI (sex) - **02/out**

Parte 2

- Prova 2 - P2
  - Eng. Soft T03 (seg) - **16/nov**
  - Eng. Soft T02 (sex) - **13/nov**
  - SI (sex) -**13/nov**

Aulas de dúvida na semana **anterior** às provas

Avaliação

Seminário

15% da nota

Apresentações sobre temas de sistemas operacionais

Recuperação

- Exame sobre o conteúdo da disciplina
  - Eng. Soft T03 (seg) - **30/nov**
  - Eng. Soft T02 (sex) -**04/dez**
  - SI (sex) - **04/dez**
- Substitui sua menor nota
- Nota final truncada em 5,0

Requisitos

- Ter feito ao menos uma das provas
  - Prova entregue em branco não conta!
- Ter entregue o seminário
- Não ter extrapolado o limite de faltas

Média Final = P1 * 0,425 **+ P2** * 0,425 **+ S** * 0,15

Frequência

- Necessário **75% de frequência**
  - Chamadas feitas ao final da aula
- Atrasos e saídas antecipadas:
  - Tolerados em dias normais
  - Sem entrada após alguém sair em dia de prova
- Atestados devem passar pela secretaria
  - Atestados de um dia não são válidos - usem seus 25% de faltas

Perguntas?



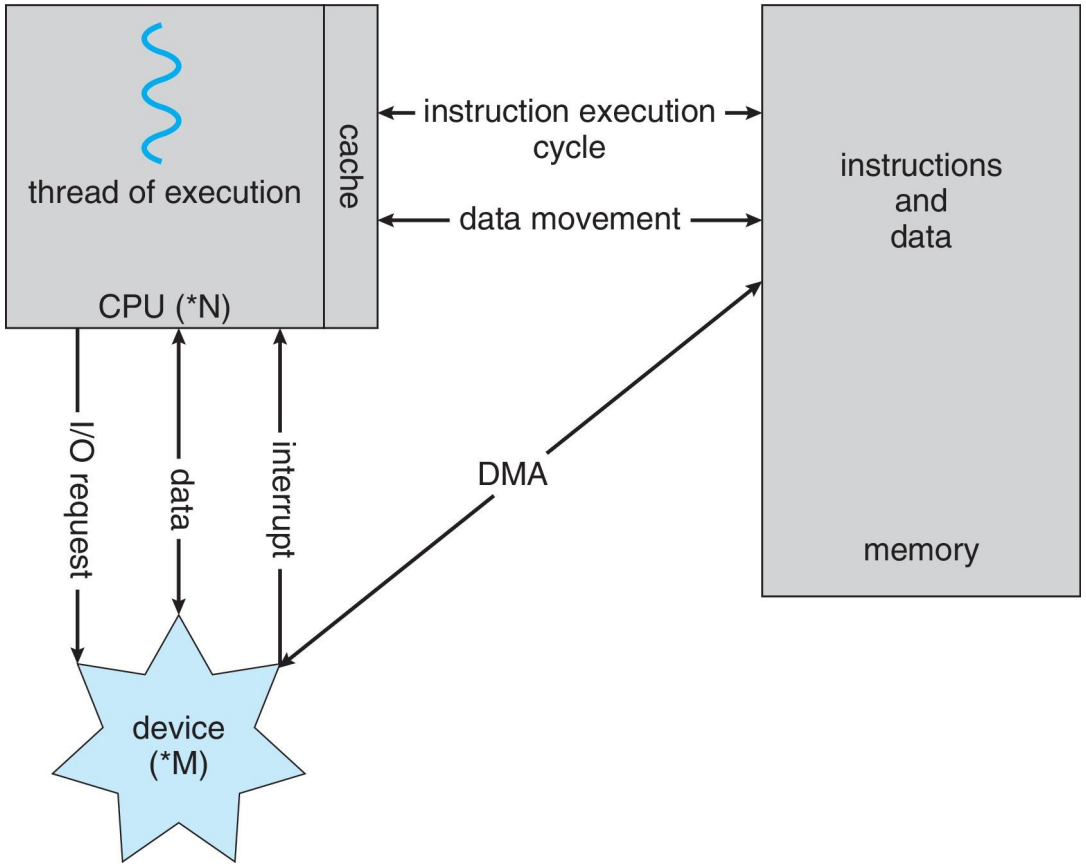
Nesta aula...

- Como definimos um sistema operacional
- Principais funcionalidades e responsabilidades
- Conceitos importantes:
  - **Kernel**
  - **Chamada de Sistema (Syscall)**





Estrutura do Computador Moderno



A von Neumann architecture





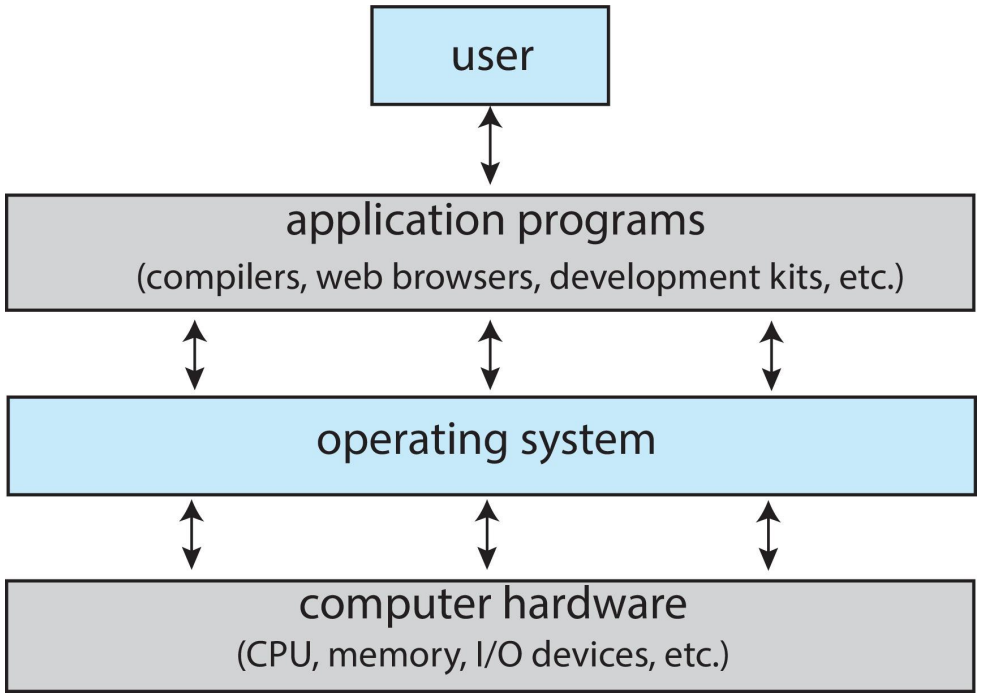
O que é um Sistema Operacional?

- Um programa que age como intermediário entre o usuário e o hardware
- Objetivos de um SO:
  - Executar programas e facilitar a solução de problemas do usuário
  - Manter o sistema fácil de ser utilizado
  - Usar o hardware do computador de maneira eficiente





Visão Abstrata de um computador







O que Sistemas Operacionais fazem?

- Depende do ponto de vista…
- Usuários querem conveniência, **facilidade** e **bom desempenho**
  - Não se importam com **utilização de recursos**
- Computadores compartilhados como **mainframes** precisam agradar todos os usuários
  - SO é um **alocador de recursos** e **programa de controle**
- Usuários de sistemas dedicados como **workstations** possuem recursos próprios mas frequentemente usam recursos compartilhados de **servidores**
- Dispositivos móveis tem poucos recursos, focados em usabilidade e tempo de bateria
- Alguns computadores têm pouca ou nenhuma interface de usuário
  - Executam sem intervenção do usuário





Definição de Sistemas Operacionais

- Não há uma única definição aceita
- "Tudo que o fornecedor entrega com o sistema" é uma boa aproximação
- "O programa executando o tempo todo no computador" é o **kernel**
- O restante é:
  - ***Programa de sistema*** (vem com o SO, mas não é parte do kernel), ou
  - Um ***programa de aplicação***, não associado ao SO
- SOs modernos de propósito geral e dispositivos móveis incluem ***middleware*** – conjunto de frameworks que provê serviços adicionais para desenvolvedores de aplicações
  - DirectX
  - NET Framework
  - *libc, GTK, Qt…*





O Kernel

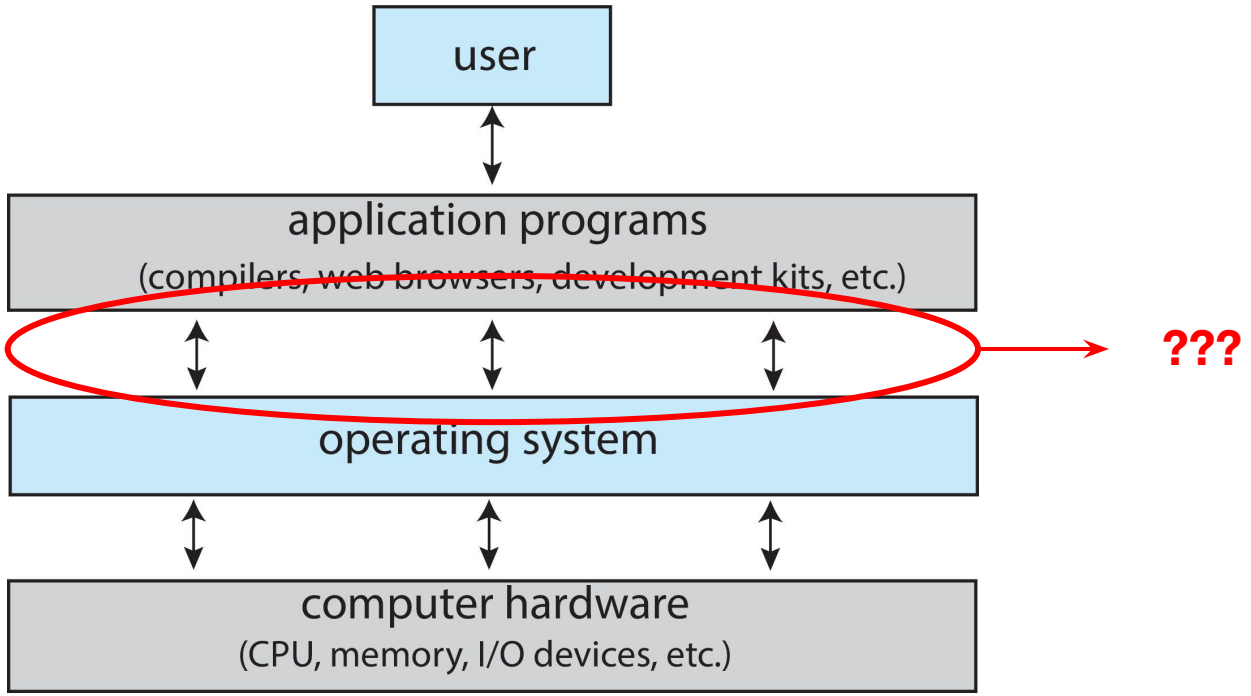
- Núcleo do Sistema Operacional
- Responsável pelas decisões centrais
  - Gerenciar hardware (CPU, memória, dispositivos)
  - Executar/remover processos
  - “Ponte” entre software e hardware
- Foco do estudo da disciplina (quando falamos de SO, falamos do kernel)
- Exemplo: Kernel Linux
  - Android
  - Roteadores
  - Embarcados
  - Backbone da internet
  - Top 500







Visão Abstrata de um computador



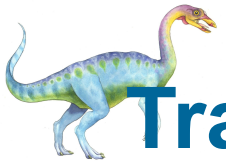




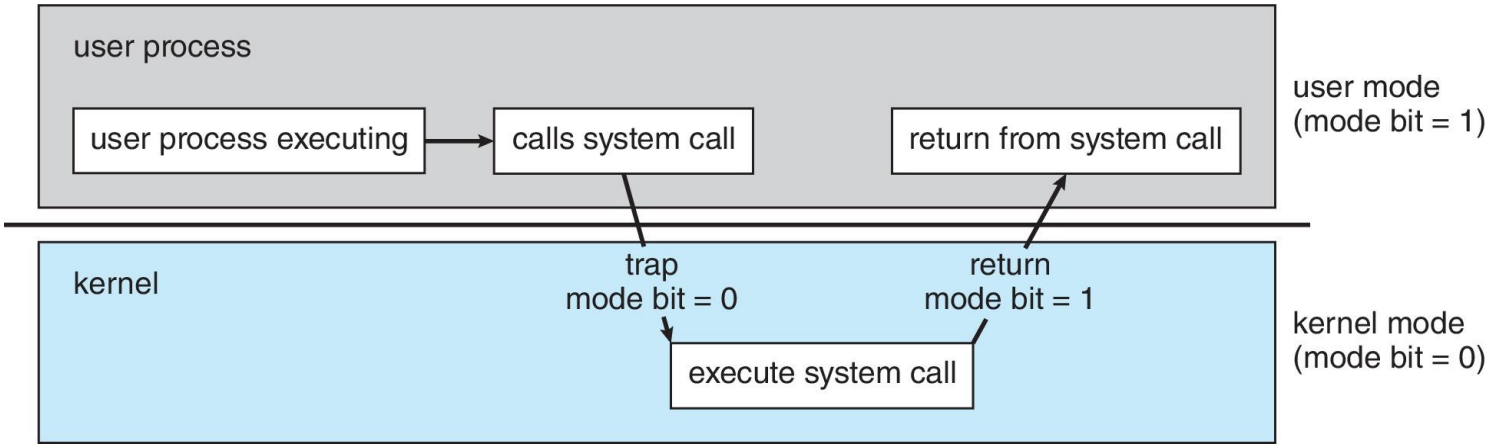
Modo de operação Dual

- Modo de operação **dual** permite ao SO proteger a si mesmo e componentes do sistema
  - **Modo usuário** e **modo kernel**
- **Bit de modo** providenciado pelo hardware
  - Consegue diferenciar quando o sistema está executando código de usuário ou kernel
  - Código de usuário □ bit de modo é “user”
  - Código do kernel □ bit de modo é “kernel”
- Como garantimos que o usuário não seta o bit de modo para kernel?
  - Transições de modo (**system call**) controlam o bit
- Algumas instruções são **privilegiadas**, executadas apenas no modo kernel





Transição de Modo - Chamada de Sistema







Gerenciamento de Processos

- Um processo é um programa em execução.
  - Unidade de trabalho
  - Programa é uma ***entidade passiva;*** processo é uma ***entidade ativa***.
- Processo precisa de recursos
  - CPU, memória, I/O, arquivos.
  - Dados de inicialização





Gerenciamento de Memória

- Para executar um programa, parte/todas as instruções precisam estar em memória
- Parte/todos os dados do programa precisam estar em memória
- Gerenciamento de memória determina o que está em memória e quando
  - Otimizando uso de CPU e tempo de resposta do computador ao usuário





Proteção e Segurança

- **Proteção** – mecanismos para controlar o acesso de processos ou usuários a recursos definidos pelo SO
- **Segurança** – defesa do sistema contra ataques externos
  - denial-of-service, vírus, roubo de identidade, roubo de serviço
- Sistemas controlam usuários, determinando quem pode fazer o que
  - Identidade de Usuário (**user ID**) inclui nome e número associado, um por usuário
  - Associa arquivos e processos controlados pelo usuário
  - Identificador de grupo (**group ID**) permite a definição e controle de conjuntos de usuários, associados a processos e arquivos
  - **Escalonamento de privilégios** permite a usuários alterarem seu ID para obter maior acesso/controle do sistema





Sistemas Operacionais Open-Source/Livre

- Sistemas operacionais disponíveis em formato código-fonte no lugar de binários **proprietários** e **código-fechado**
- Contrários a **proteção de cópia** e movimento **Digital Rights Management** (**DRM**)
- Iniciado pela **Free Software Foundation** (**FSF**), com a licença **GNU Public License** (**GPL**)
  - Software livre e softweare open-source são ideías diferentes construídas por grupos de pessoas diferentes constituídas por grupos de pessoas diferentes
    - **https://www.gnu.org/philosophy/open-source-misses-the-point.en.html**
- Exemplos incluem **GNU/Linux** e **BSD UNIX** (incluindo o núcleo do **Mac OS X**), e muitos outros





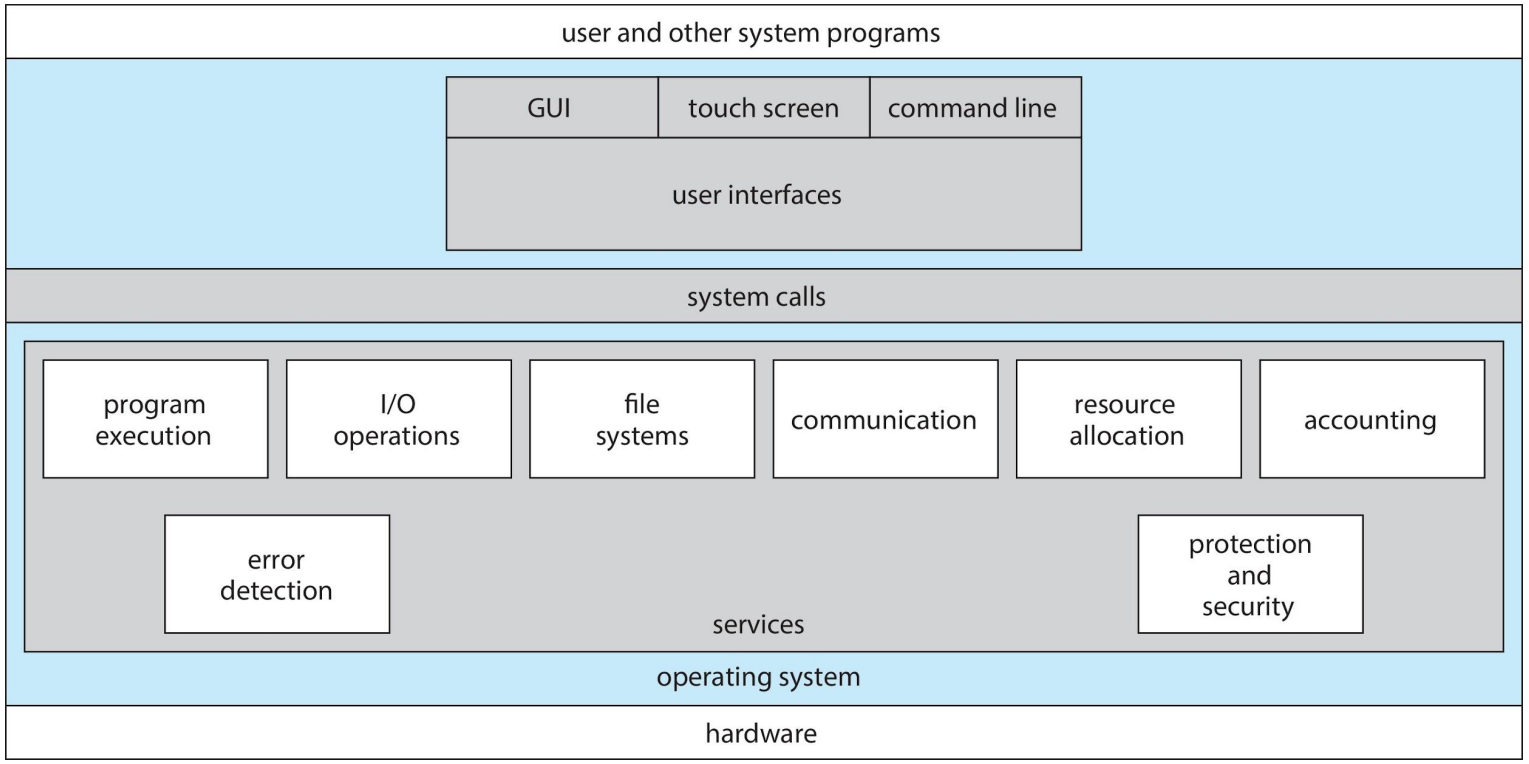
Serviços do Sistema Operacional

- Sistemas operacionais propõem um ambiente de execução para uso de programas e usuários
- Conjunto de serviços providos pelo sistema operacional:
  - **Interface de Usuário** - Quase todos os sistemas operacionais possuem.
    - **Linha de Comando** (**CLI**), **Interface Gráfica** (**GUI**), **touch-screen**
  - **Execução de programa**
  - **Operações E/S**
  - **Manipulação do sistema de arquivos**





Serviços do Sistema Operacional







Chamadas do Sistema

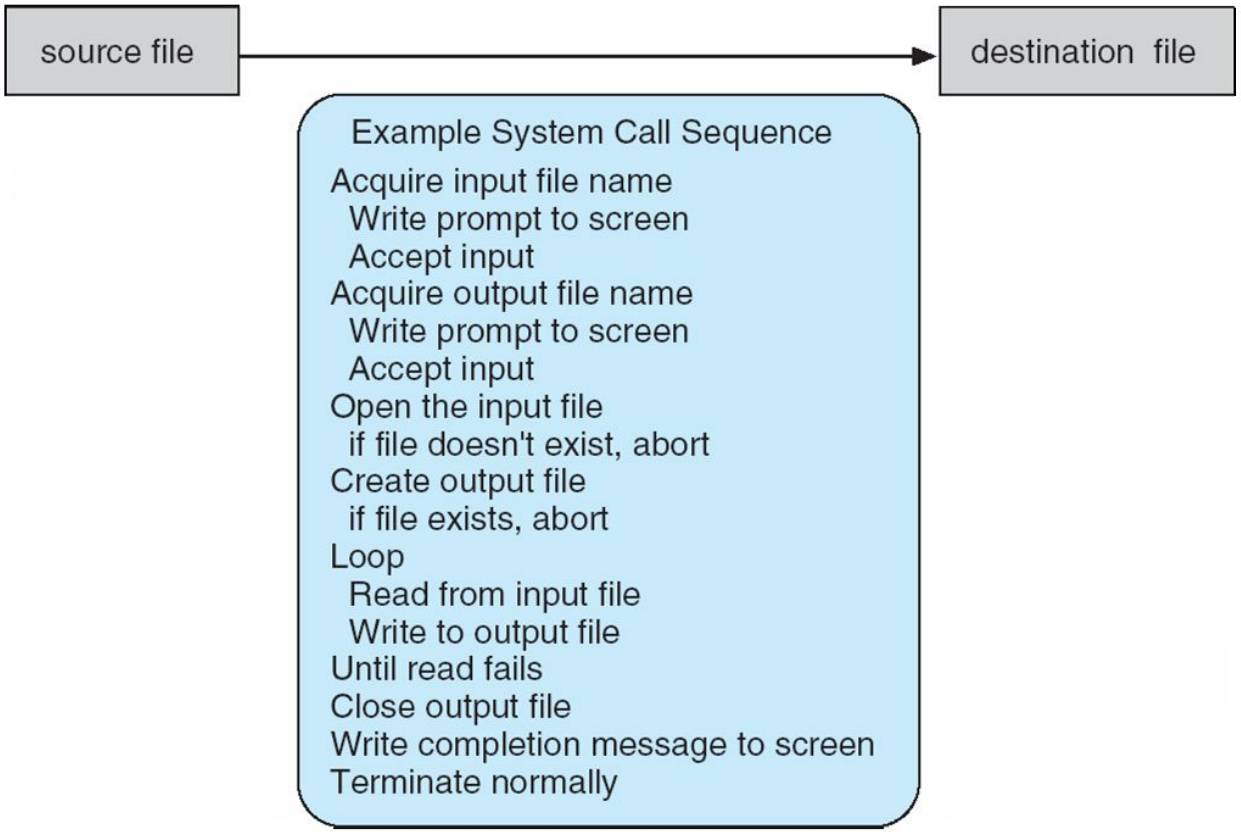
- Interface de programação para os serviços providos pelo SO
- Comumente escritas em linguagens de alto nível (C ou C++)
- Acessada por programas por meio de uma **Application Programming Interface** (**API**), não chamadas diretas a *syscall*
- Interfaces mais comuns:
  - Win32 API (Windows)
  - POSIX API (UNIX, Linux, Mac OS X)
  - Java API (Java JVM)





Exemplo de Chamadas de Sistema

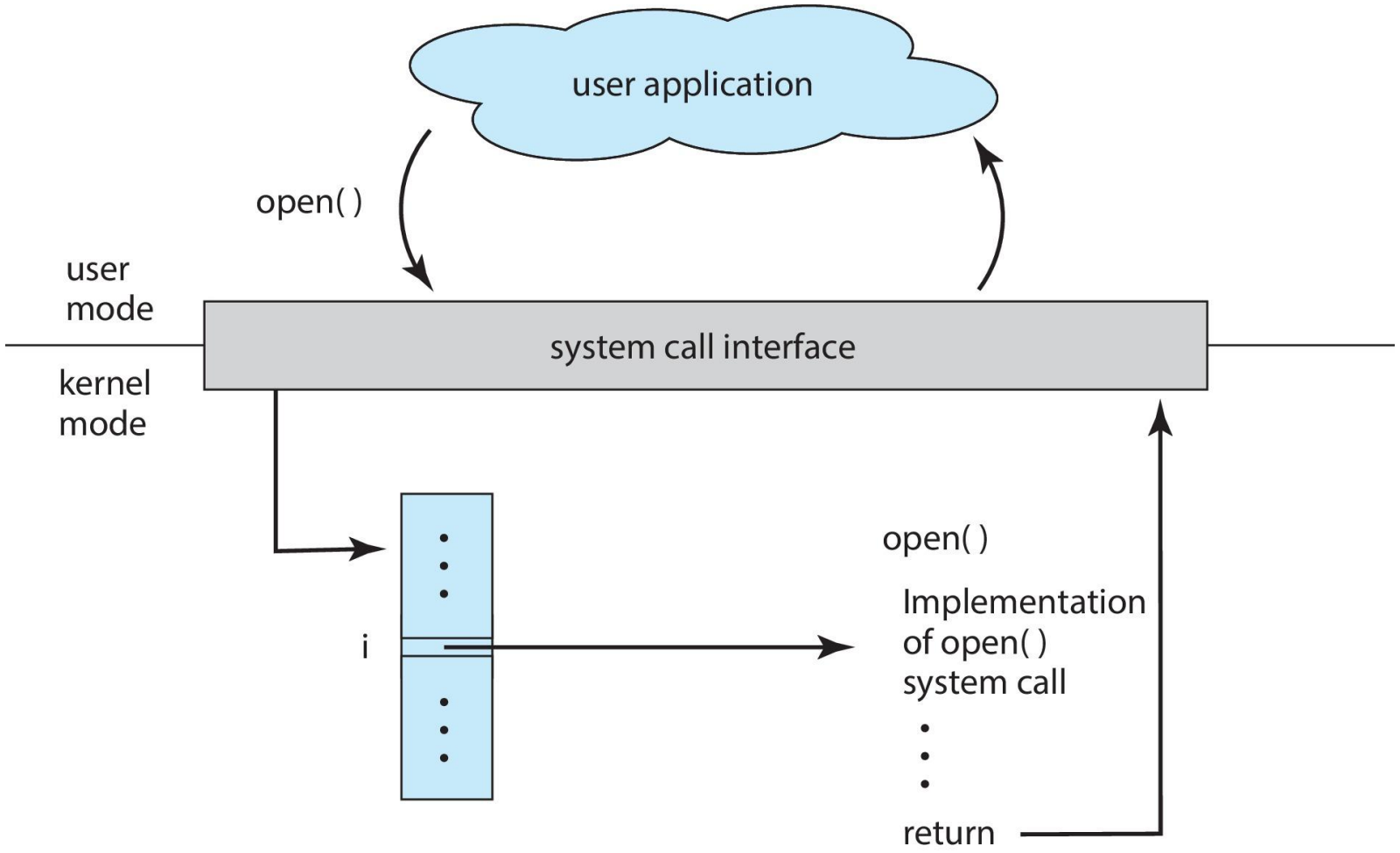
- Sequência de chamadas de sistema para copiar o conteúdo de um arquivo para outro







API – Relação Chamada de Sistema/SO







Porque aplicações são específicas a um SO

- Aplicações compiladas para um sistema não são executáveis em outro sistema
- Cada SO propõe suas próprias *syscalls*
  - Formatos de arquivo, etc.
- Aplicações podem ser multi-plataforma (SO)
  - Linguagens interpretadas como Python e Ruby tem interpretadores para vários SOs
  - Aplicações escritas para linguagens que contém máquinas virtuais (Java - JVM)
  - Aplicações que usam linguagens com compiladores/bibliotecas multiplataforma, compiladas para seu SO desejado (C, C++, Rust)





Implementação de um SO

- Muita variação
  - Inicialmente escritos em assembly
  - Então linguagens como Algol, PL/1
  - Agora C, C++, Rust
- Muitas vezes uma mistura de diversas linguagens
  - Níveis mais baixos em assembly
  - Parte principal em C
  - Programas em C, C++, linguagens scriptadas como PERL, Python, shell scripts
- Linguagem de alto nível = maior **portabilidade**
  - Mais lento





Estrutura de um Kernel

- SOs de propósito geral são programas **enormes**
- Várias maneiras de estruturar um
  - Simples – MS-DOS
  - Monolítica – UNIX
  - Em camadas – an abstraction
  - Microkernel – Mach





Estrutura Monolítica – UNIX

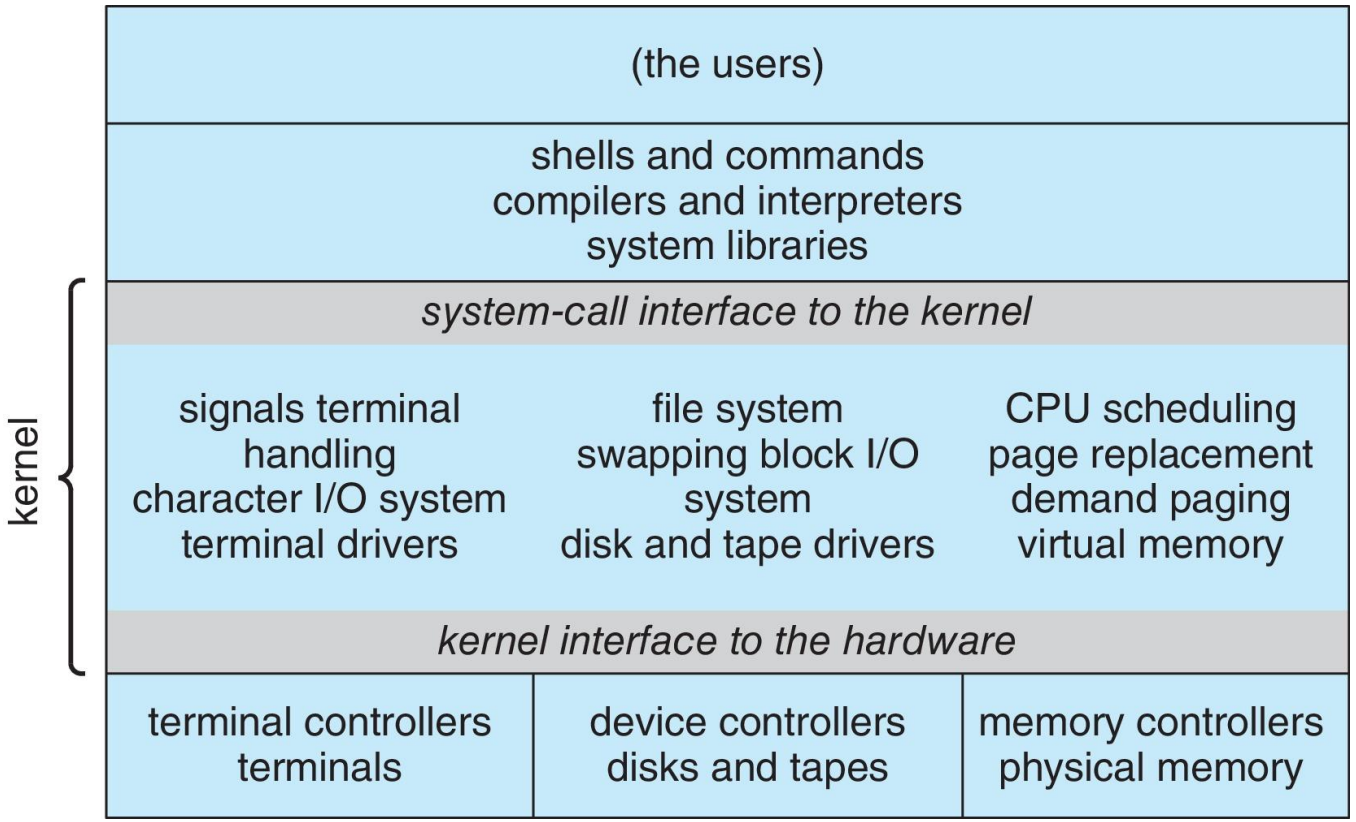
- UNIX – limitado pelas funcionalidades do hardware, o UNIX original tinha estrutura limitada.
- UNIX consiste de duas partes:
  - Programas de sistema
  - Kernel
    - Tudo abaixo da interface de *syscalls* e acima do hardware
    - Provê o sistema de arquivos, escalonamento da CPU, gerenciamento de memória, etc…
    - Uma quantidade **enorme** de funções para um nível





Estrutura Tradicional UNIX

Não é simples, mas não possui muitas camadas

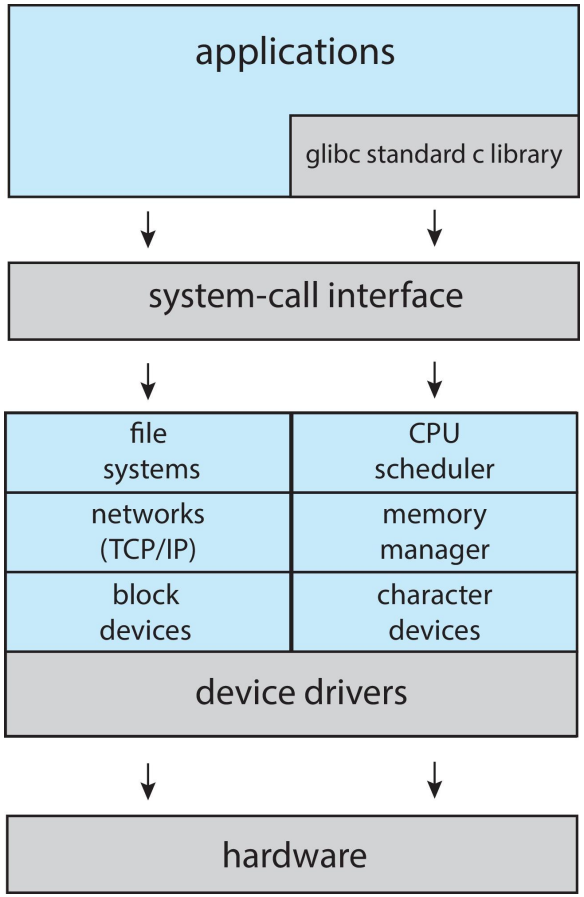






Estrutura de Sistemas Linux

Monolítica + Modular







Microkernels

- Deixa o máximo possível em espaço de usuário
- **Mach** é um exemplo de **microkernel**
  - O kernel do Mac OS X (**Darwin**) é baseado em Mach
- Comunicação ocorre entre módulos de usuário utilizando *passagem de mensagens*
- Benefícios:
  - Mais fácil de entender
  - Mais fácil de portar para outras arquiteturas
  - Mais seguro
- Malefícios:
  - Overhead de comunicação entre espaço de usuário e espaço de kernel





Estrutura de Microkernel

